

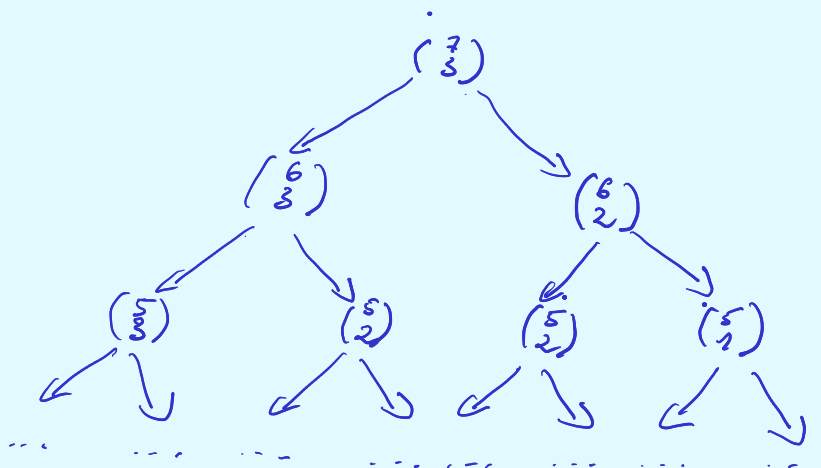
Przykład:

Dane: $n, k, m \in \mathbb{N}$
 Wynik: $\binom{n}{k} \bmod m$

Rozważania:

1) $\binom{n}{k} = \frac{n!}{k!(n-k)!}$

2) $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$ - problem - złożona rekurencja
 1, gdy $k=0$



T:

n \ k	0	1	2	3
0	1			
1	1	1	1	
2		2	3	1
3			6	4
4				
5				
6				
7				

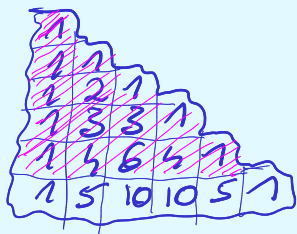
SPAMIĘTYWANIE
 (nie ma tego w rekursji)

$npk(n, k):$

if ($n == k$ || $k == 0$) return 1
return $npk(n-1, k-1) + npk(n, k-1)$

if ($T(n-1, k-1) == ?$)
 $T[n-1, k-1] = npk(n-1, k-1)$
if ($L(n-1, k) == ?$)
 $T[n-1, k] = npk(n-1, k)$
return $T[n-1, k-1]$

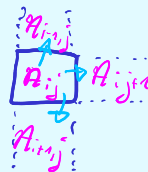
Łojko i pasce



- sprawdzamy tylko ostatni wiersz

Przykład:

Dane: tablica A $n \times n$ liczb naturalnych

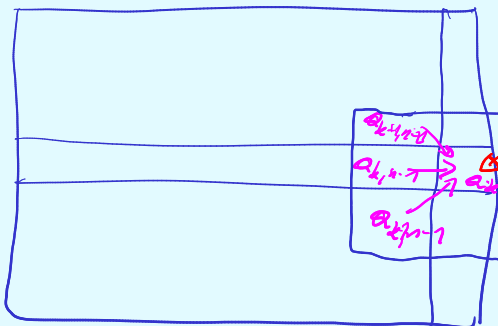


Zadanie: znaleźć najtańszą drogę wiedząc z pierwszej kolumny do ostatniej.

droga $i_1, i_2, i_3, \dots, i_n$ wiedzie przez pola $(i_j, j) \forall j = 1 \dots n$

Koszt drogi $\sum_{j=1}^n A[i_j, j]$

~~Pomysł -1 (naivny) - sprawdzić wszystkie drogi~~



jeżeli kończy się tutaj jej wartość wynosi minimum ($\min(\text{najtańsza}(A[k-1, n-1]), \text{najtańsza}(A[k, n-1]), \text{najtańsza}(A[k+1, n-1])) + A[k, n]$

Podproblemy:

najtańsza ścieżka od pierwszej kolumny do komórki (i, j)

liczba podproblemów =
liczba kolumn
x liczba wierszy

7	5	1	8	
1	8	1	3	
4	6	2	5	
2	3	4	9	
6	7	7	6	

PRZYKŁAD (Problem LCS - longest common subseries)

Dane: $X: x_1, x_2, \dots, x_n$
 $Y: y_1, y_2, \dots, y_n$

podciąg powstaje przy usunięciu elementów z ciągu.

Znaleźć $Z \in 2_1, \dots, 2_k$, t.j.:

1. Z jest podciągnięciem X oraz Y
2. k - możliwie największe.

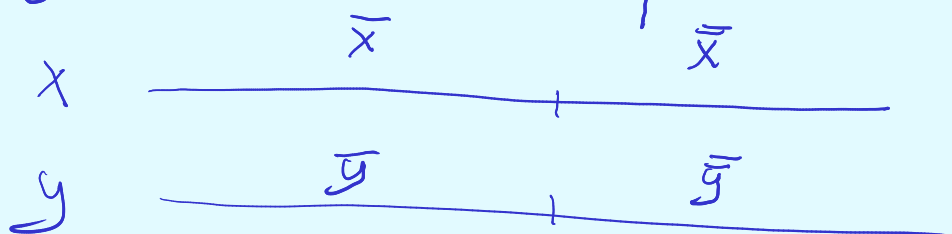
Zbiór 2-tórek spełniających te warunki oznaczamy $LCS(X, Y)$

np.

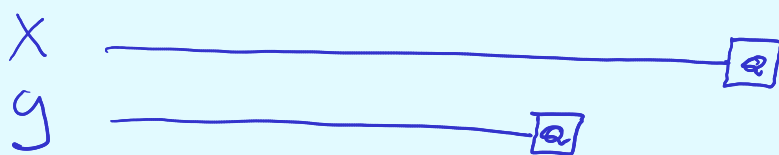
$X = \text{BARAKUDA}$
 $Y = \text{BARBAKAN}$

Wspólne podciągi:

B, BARAK, BAKA,
BARAKA

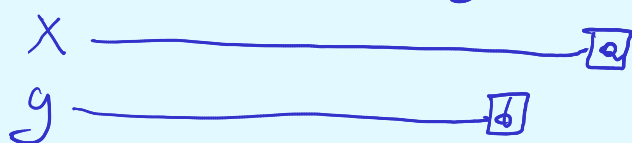


SPOSTRZĘZENIE



Jeśli $x_n = y_m$, to
 $\forall Z \in LCS(X, Y)$
 $z_k = a$

A co jeśli $x_n \neq y_m$?



Istnieje $LCS(X, Y)$,
który nie kończy się na x_n
lub istnieje $LCS(X, Y)$,
który nie kończy się na y_m .

Ograniczamy się chwilowo do obliczenia długości ciągów w $LCS(X, Y)$.

Podproblemy: szukanie LCS dla $x_1 \dots x_i$ oraz $y_1 \dots y_j$

Niech d_{ij} = długości ciągów $LCS(x_1 \dots x_i, y_1 \dots y_j)$

$$d_{ij} = \begin{cases} 0 & i=0 \text{ lub } j=0 \\ d_{i-1, j-1} + 1 & x_i = y_j \\ \max(d_{i-1, j}, d_{i, j-1}) & \text{wpp.} \end{cases}$$

d.

	i \ j	0	B	A	C	A
0	0	0	0	0	0	0
A	1	0	$\max(0,0)$ 0	0+1 1	$\max(0,1)$ 1	0+1=1
B	2	0	0+1 1	$\max(1,0)$ 1	$\max(0,1)$ 1	1
A	3	0	1	1+1 2	2	1+1=2
K	4	0	1	2	2	2

$$X = \text{ABAK}$$

$$Y = \text{BACB}$$

Skomplikacja czasowa:

$$\underbrace{O(nm)}_{\substack{\uparrow \\ \text{tworzenie} \\ \text{tablicy } d}} + \underbrace{O(n+m)}_{\substack{\uparrow \\ \text{odczyt} \\ \text{podciągu}}}$$

Pamięciowa $O(nm)$

Do wypełnienia tablicy d pole d_{nm} będzie zawierać długość najdłuższego wspólnego podciągu. Jego rekonstrukcję przeprowadzamy następująco:

1. Zaczynamy z pustym podciągiem s

A $\begin{matrix} & B \\ & d_{i,j} \\ d_{i,j-1} & d_{i,j} \end{matrix} \rightarrow$ idziemy do większego a pól $d_{i,j-1}, d_{i-1,j}$

A $\begin{matrix} & A \\ d_{i,j} \\ d_{i,j} \end{matrix} \rightarrow$ idziemy do $d_{i-1,j-1}$ i dopisujemy A do s od lewej
 $i, j > 0$

x_i $\begin{matrix} y_j \\ d_{i,j} \end{matrix} \rightarrow$ jeśli $y_j = x_i$ dopisujemy x_i do A w przeciwnym razie nic nie dopisujemy. Następnie kończymy procedurę i szukamy podciągu to s .

Pytanie. Czy można poprawić złożoność pamięciową?

Zastosowanie: Porównywanie łańcuchów DNA.